

L40: Tabular Q-Learning for Queuing Control

Simulation Without a Black Box

Outline

The Q-Function

Q-Learning Update Rule

Epsilon-Greedy Exploration

State Discretization

Code: Q-Table Implementation

Results and Limitations

Key Takeaways

The Action-Value Function $Q(s, a)$

Definition

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right]$$

The expected *cumulative discounted return* when taking action a in state s , then following policy π .

Optimal Q-function:

$$Q^*(s, a) = \max_\pi Q^\pi(s, a)$$

Optimal policy: $\pi^*(s) = \arg \max_a Q^*(s, a)$

Once Q^* is known, no planning is needed — just act greedily.

Queuing control example:

State q	Action a	$Q^*(s, a)$
$q = 0$	open server	-1.2
$q = 0$	keep closed	-0.8
$q = 5$	open server	-2.1
$q = 5$	keep closed	-8.4

Learned values guide staffing decisions.

Q-Learning Update Rule

Watkins' Q-Learning (1989)

After observing transition (s, a, r, s') :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{TD target}} - Q(s, a) \right]$$

Components:

- $\alpha \in (0, 1]$: learning rate
- $r + \gamma \max_{a'} Q(s', a')$: one-step Bellman target
- $r + \gamma \max_{a'} Q(s', a') - Q(s, a)$: TD error δ

Off-Policy Property

Q-learning uses $\max_{a'}$ regardless of what action the agent actually takes next. This makes it *off-policy* — the learned Q converges to Q^* even under exploratory behavior.

Convergence Guarantee

Q-learning converges to Q^* with probability 1 if:

- All (s, a) visited infinitely often
- $\sum_t \alpha_t = \infty$; $\sum_t \alpha_t^2 < \infty$

ϵ -Greedy Exploration

The exploration–exploitation dilemma:

- **Exploitation:** choose $\arg \max_a Q(s, a)$ — maximise known reward
- **Exploration:** choose a random action — discover better regions

ϵ -greedy policy:

$$a = \begin{cases} \text{random action} & \text{with prob. } \epsilon \\ \arg \max_a Q(s, a) & \text{with prob. } 1 - \epsilon \end{cases}$$

ϵ **decay:** start at 1.0, decay to 0.01 over training.

```
def choose_action(Q, s, eps, n_actions):  
    if np.random.rand() < eps:  
        return np.random.randint(n_actions)  
    return np.argmax(Q[s, :])
```

```
# Decay schedule  
eps = 1.0  
for episode in range(N_EP):  
    eps = max(0.01, eps * 0.995)  
    ...
```

Warning

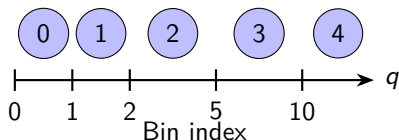
Decaying ϵ too fast $\Rightarrow$ premature convergence to a suboptimal policy. Always plot the episode return while decaying.

State Discretization for Queue Length

Problem: Queue length $q \in \{0, 1, 2, \dots\}$ is countably infinite. Table $Q[s, a]$ cannot be infinite.

Solution: bin the state.

- Define bins: $[0], [1], [2-4], [5-9], [10+]$
- Map q to bin index $b(q) \in \{0, 1, 2, 3, 4\}$
- Table size: $|\mathcal{S}| \times |\mathcal{A}|$ (e.g., $5 \times 3 = 15$ entries)



```
bins = [0, 1, 2, 5, 10, np.inf]
def discretize(q):
    return np.digitize(q, bins) - 1
```

Tradeoff

Coarse bins $\Rightarrow$ information loss, policy may be suboptimal. Fine bins $\Rightarrow$ table grows, requires more experience to fill.

Q-Table as a NumPy Array

```
import numpy as np
from simdes.envs import QueueGymEnv

env = QueueGymEnv(arrival_rate=0.8, service_rate=1.0, seed=42)
n_states, n_actions = 5, 2      # bins x {0:close, 1:open server}
Q = np.zeros((n_states, n_actions))
alpha, gamma, eps = 0.1, 0.99, 1.0

for ep in range(5_000):
    s, _ = env.reset(); done = False
    while not done:
        a = int(np.random.rand() < eps) * np.random.randint(n_actions) \
            + int(np.random.rand() >= eps) * np.argmax(Q[s])
        s_next, r, done, _, _ = env.step(a)
        Q[s, a] += alpha * (r + gamma * np.max(Q[s_next]) - Q[s, a])
        s = s_next
    eps = max(0.01, eps * 0.995)
```

Results: Learned Policy vs. Fixed Policy

Comparison on QueueGymEnv ($\rho = 0.8$, 500 test episodes):

Policy	Mean W_q	Std
Always open ($c=1$)	4.01	0.62
Always closed ($c=0$)	∞	—
Threshold ($q \geq 3 \Rightarrow$ open)	2.84	0.51
Q-learning (learned)	2.31	0.48

Limitations of tabular Q-learning:

- **Curse of dimensionality:** table size = $|\mathcal{S}| \times |\mathcal{A}|$; grows exponentially with features.
- **Discretization error:** binning loses information.
- **Sample efficiency:** needs many episodes to fill the table.

When to move to deep RL

When $|\mathcal{S}|$ cannot fit in memory (e.g., multi-server, multi-product, time-varying arrivals), use a neural-network approximation.

Key Takeaways

L40 Summary

1. $Q(s, a)$ is the expected cumulative reward from (s, a) ; once known, the optimal policy is greedy over Q^* .
2. The Q-learning update is $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ — off-policy, converges to Q^* under mild conditions.
3. ϵ -greedy exploration balances discovery and exploitation; decay ϵ slowly and monitor episode returns.
4. Discretize continuous state variables into bins — coarser bins reduce table size at the cost of policy quality.
5. Tabular Q-learning is the right starting point; move to function approximation when the state space cannot be enumerated.

Next: L41 — Wrapping SimPy as a Gymnasium environment.